Aplikacje Internetowe
Podstawy Technologii Internetowych

Dr inż. **Wikarek Jarosław**
Katedra Informatyki Stosowanej
e-mail: j.wikarek@tu.kielce.pl
pokój: d408

Wydział Elektrotechniki, Automatyki i Informatyki

Protokoły

Protokół to zestaw reguł umożliwiających porozumienie się.

PROTOKÓŁ HTTP

Dla serwisów WWW podstawę stanowi protokół **HTTP** – jednokierunkowy, bezpołączeniowy protokół przesyłania hipertekstu (Hypertext Transfer Protocol)

Protokoły – HTTP

Historia

- Pierwotną wersją HTTP był prosty protokół http v0.9 do przesyłania surowych danych przez Internet.
- HTTP wersja 1.0 udoskonala go poprzez umożliwianie wysyłania wiadomości w formacje MIME (Multipurpose Internet Mail Extensions – standard umożliwiający kodowanie i przesyłanie dowolnych typów danych)
- HTTP wersja 1.1 poprawiono użycie serwerów proxy, tajność składowania danych, wirtualnych hostów itp.
- HTTP wersja 2.0 W dużym uproszczeniu HTTP/2 jest specyfikacją tego, jak powinien komunikować się serwer z przeglądarką w taki sposób, aby dane były przesyłane w jak najszybszy sposób.

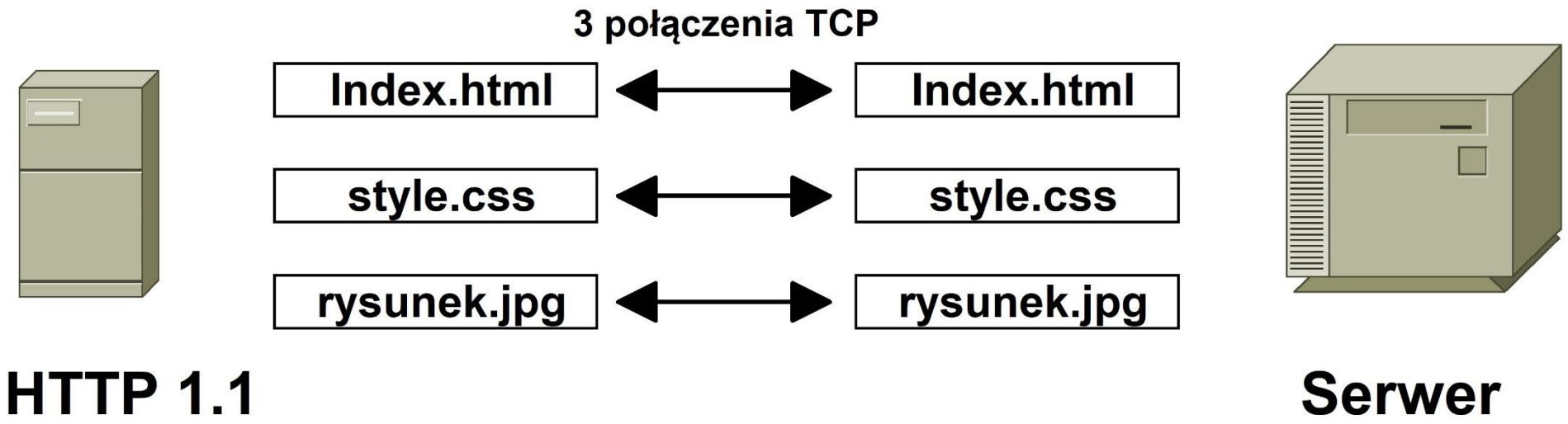
Protokoły – HTTP

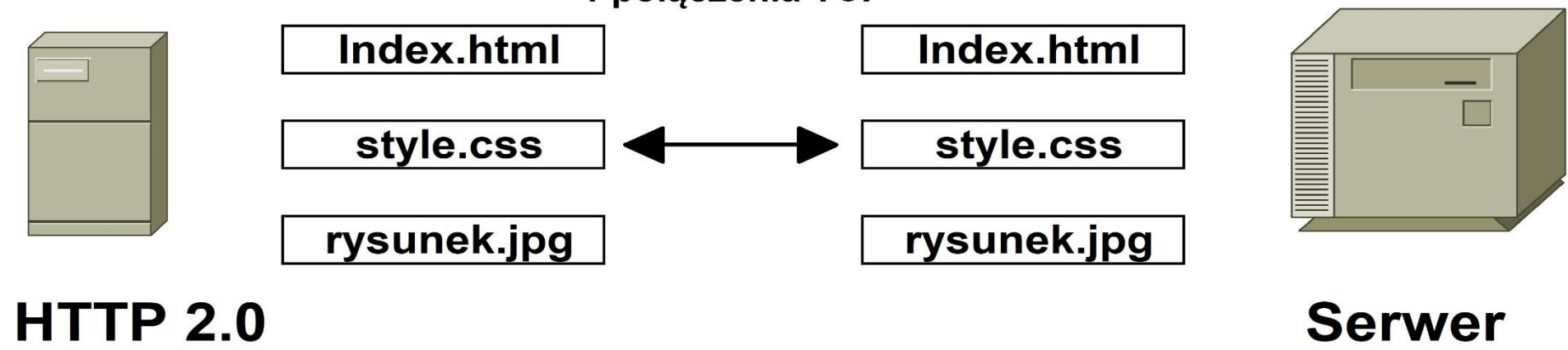
Historia

- HTTP/3 oparty na protokole QUIC (który z kolei oparty jest na UDP) całkowicie eliminuje blokowanie nagłówków. Protokół HTTP/3 umożliwia jeszcze lepsze multipleksowanie. Zgubienie jakiegokolwiek pakietu danych nie wiąże się z koniecznością odświeżenia połączenia.

Protokoły – HTTP

Wersja 1.1 a 2.0





Protokoły – HTTP

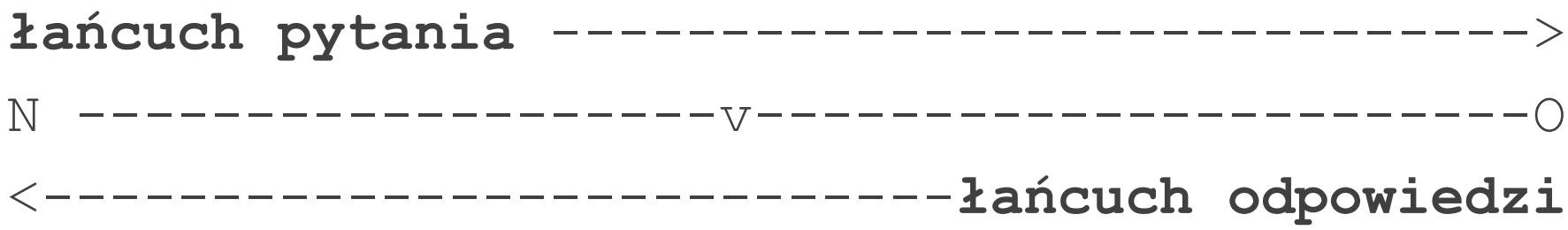
Wersja 1.1 a 2.0

- HTTP/2 umożliwia wykonanie wielu zapytań do serwera jednocześnie, tzn. nie kolejkują się, tylko realizują jednocześnie w tym samym czasie. Zatem w przypadku tego protokołu możliwe jest, że jednocześnie pobierana się plik HTML i CSS.
- W przypadku tego protokołu wykorzystuje się polecenia binarne. Sprawia to, że mniejsze jest ryzyko błędów. To również wpływa na prędkość wczytywania się serwisu.
- HTTP/2 wymaga posiadania certyfikatu SSL.
- HTTP/2 nie jest obsługiwany przez każdy serwer.

Protokoły – HTTP

Zasada działania

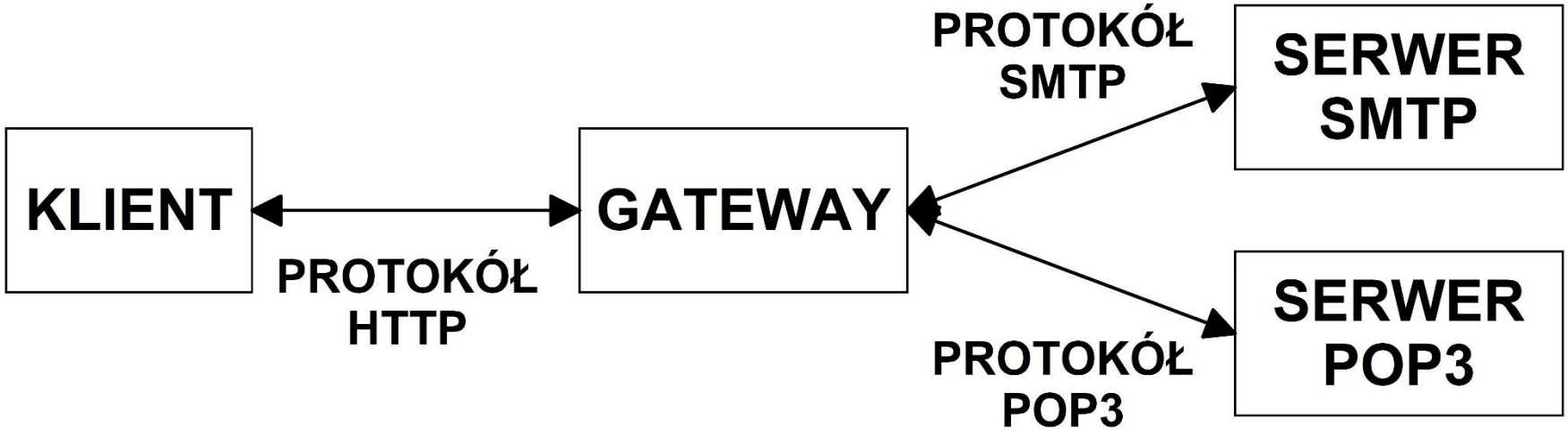
- **Podstawowa** – Większość komunikacji HTTP jest inicjowana przez klienta i składa się z prośby o złożone zasoby na źródłowym serwerze. W najprostszym przypadku może to być zrealizowane przez pojedyncze połączenie (v) pomiędzy użytkownikiem(U) a serwerem źródłowym (O)



Protokoły – HTTP

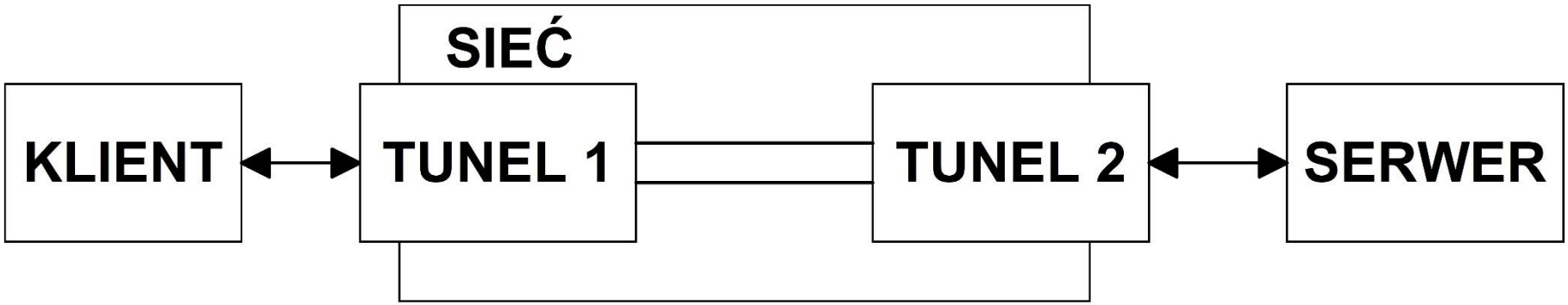
Zasada działania

- Bramka



Zasada działania

- Tunel



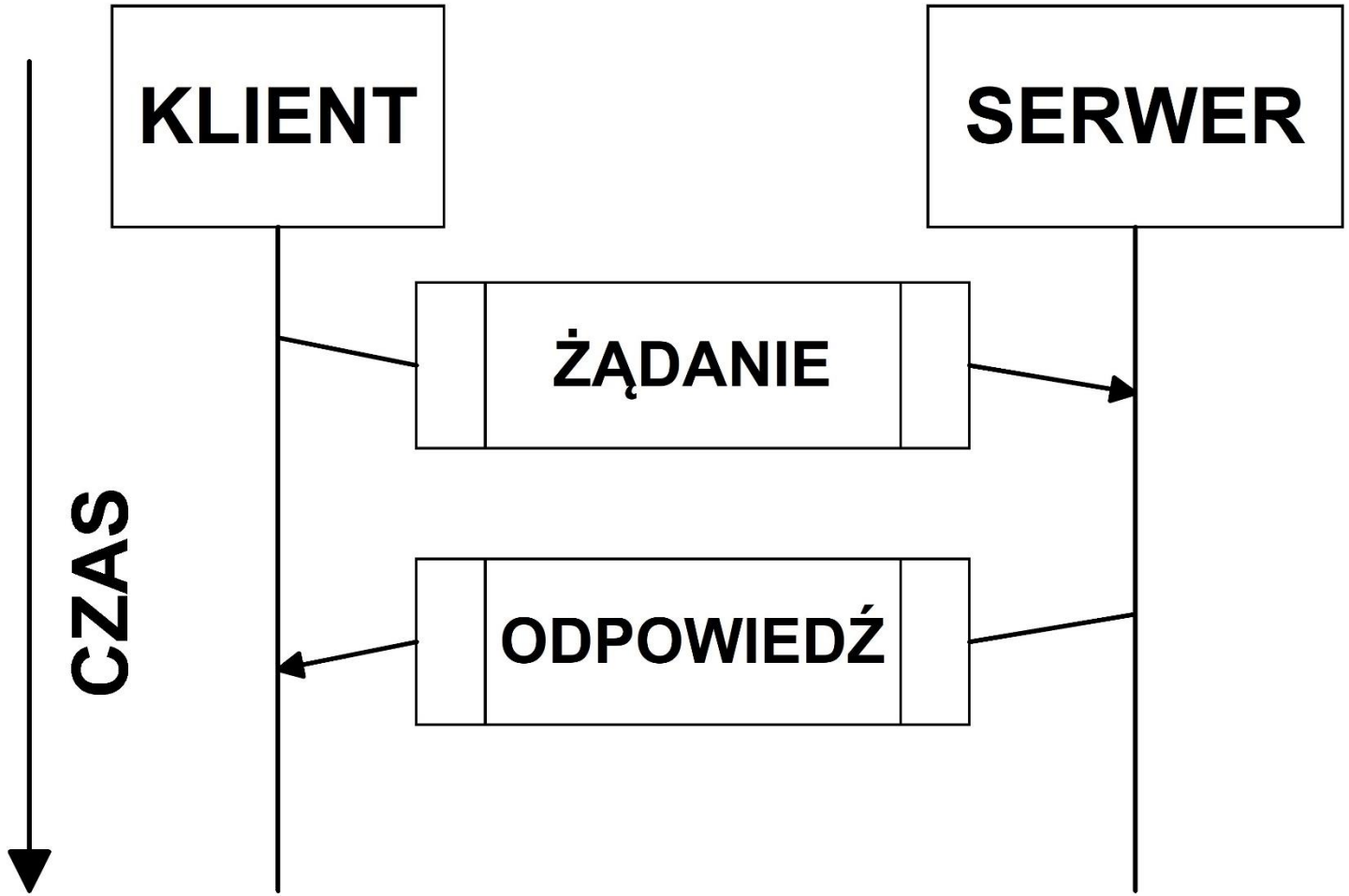
Zasada działania

- Proxy



PROTOKÓŁ HTTP 1.1

Zasada działania



Protokoły – HTTP

Budowa żądania

```
metoda URL wersja-HTTP
dodatkowe pola nagłówka
<pusta linia>
dane
```

Protokoły – HTTP

Metody

- **GET** – Wysłanie tego żądania oznacza, iż klient chciałby otrzymać zasób z serwera, czyli może to być zawartość pliku, jak również dane wyjściowe wywołanego programu.
- **HEAD** – Oznacza, iż klient chciałby otrzymać informacje na temat dokumentu, a właściwie nagłówka, gdyż treść sama w sobie nie jest mu w ogóle potrzebna. Metoda ta pozwala na uzyskanie między innymi informacji o:
  - ✓ Jaka jest data ostatniej modyfikacji, wykorzystywanej w mechanizmach buforowania danych.
  - ✓ Rozmiarze dokumentu, jego typie oraz typie serwera.

Protokoły – HTTP

Metody

- **POST** – Informuje, że klient udostępnia jakieś informacje (z możliwości tej korzysta się zazwyczaj w formularzach).
- **PUT** – Metodę tą wykorzystuje się do tego, aby klient mógł wysłać żądanie zapisania treści nowego dokumentu, bądź nowej treści dokumentu na serwerze pod wskazanym adresem.
- **DELETE** – Metoda służy do usuwania dokumentu z serwera.
- **TRACE** – Metoda żąda od serwerów pośredniczących (proxy), aby w nagłówku znajdowały się informacje o nich, co umożliwia np. znalezienie miejsca uszkodzenia lub utraty pliku.

Protokoły – HTTP

Metody

- **OPTIONS** – Dzięki tej metodzie klient ma możliwość sprawdzenia, jakie metody można jeszcze użyć przy danym dokumencie lub, jakie metody obsługuje określony serwer. W tej metodzie można zarówno podać adres URL jak i znak gwiazdki (*), który oznacza odwołanie do całego serwera.
- **CONNECT** – Metoda używamy, przy wykorzystaniu protokołów HTTPS (szyfrowana wersja protokołu HTTP) jeśli poprzez serwer pośredniczący klient chce porozumieć się z serwerem.

Protokoły – HTTP

URL

```
protokół://[użytkownik[:haslo]@]serwer
  [:nr_portu]/[ścieżka[?argumenty]]
```

~	%7e	+	%2b	&	%26
$	%24	=	%3d	*	%2a
^	%5e	#	%23		

Wersja HTTP

```
HTTP/1.1
```

Protokoły – HTTP

Budowa odpowiedzi

```
Wersja-HTTP Kod-stanu Wynik-operacji
dodatkowe pola nagłówka
<pusta linia>
dane
```

Wersja HTTP

```
HTTP/1.1
```

Protokoły – HTTP

Kod odpowiedzi

- 100-199 Informacja
- 200-299 Żądanie zostało przetworzone poprawnie
- 300-399 Żądanie zostało przeadresowane; niezbędne jest wykonanie dalszych kroków
- 400-499 Żądanie jest niekompletne
- 500-599 Wystąpił błąd w pracy serwera

Protokoły – HTTP

Dodatkowe pola

- **Nagłówek ogólny** – Opcjonalne pola ogólnego nagłówka wiadomości wykorzystywane są zarówno w żądaniach klienta, jak i odpowiedziach serwera. Pola te zawierają informacje ogólne, na przykład bieżący czas lub opis „ścieżki” połączenia sieciowego pomiędzy klientem i serwerem.
- **Nagłówek żądania** – Zawiera on dodatkowe informacje o kliencie, wykorzystywane przez serwer. Klient może tu podać serwerowi informacje o sobie i użytkowniku oraz określić formaty dokumentów, które chciałby pobrać z serwera.

Protokoły – HTTP

Dodatkowe pola

- **Nagłówek odpowiedzi** – Informuje klienta o konfiguracji serwera. Mogą się w nim znaleźć informacje o obsługiwanych metodach, żądania autoryzacji, czy też polecenie nakazujące klientowi, by spróbował wykonać daną czynność później.
- **Nagłówek zawartości** – Wykorzystuje się przy wysyłaniu treści. Znajdują się w nim informacje o tej treści, takie jak metoda kodowania, długość, typ czy też pochodzenie.

Protokoły – HTTP

Nagłówek ogólny

- Za pomocą tego pola określa się pożądane zachowanie mechanizmu buforowania danych, np. w serwerach pośredniczących.

```
Cache-Control: dyrektywa [, dyrektywa ...]
```

Dyrektywy używane w zapytaniach:

- ✓ **no-cache** Program buforujący może przechowywać kopię dokumentu, ale przed przesłaniem jej do klienta musi sprawdzić czy jest zgodna z oryginałem.
- ✓ **no-store** Informacje należy usunąć tuż po przekazaniu.
- ✓ **max-age=sekundy** Nie należy wysyłać odpowiedzi starszych niż podano to w polu sekundy.
- ✓ **only-if-cached** Nie należy pobierać nowych danych

Protokoły – HTTP

Nagłówek ogólny

- Za pomocą tego pola określa się pożądane zachowanie mechanizmu buforowania danych, np. w serwerach pośredniczących.

```
Cache-Control: dyrektywa [, dyrektywa ...]
```

Dyrektywy używane w odpowiedziach:

- ✓ **public** Dokument może być buforowany przez każdy mechanizm buforowania.
- ✓ **private** Nie może być buforowany przez mechanizm udostępniany wielu użytkownikom.
- ✓ **no-cache** Mechanizm może przechowywać kopię dokumentu, ale przed przekazaniem jej klientowi zawsze musi sprawdzić, czy nie pojawiła się nowsza wersja na serwerze.
- ✓ **no-store** Przekazywany dokument nie może być przechowywany.

Protokoły – HTTP

Nagłówek ogólny

- Do wyrażenia daty i czasu można stosować trzy formaty, przy czym zalecany to format zgodny ze specyfikacją RFC 1123: Mon, 26 May 2004 12:30:00 GMT

```
Date: format-daty
```

- Pole zawiera informacje dla serwerów pośredniczących i bramek sieciowych. Dyrektywa no-cache nakazuje pobranie żądanego dokumentu bezpośrednio z serwera docelowego.

```
Pragma: no-cache
```

- Za pomocą tego pola klient może określić inne rozpoznawane przez siebie protokoły oraz poinformować serwer, że chciałby nawiązać połączenie wykorzystujące jeden z alternatywnych protokołów

```
Upgrade: protokół/wersja
```

Protokoły – HTTP

Nagłówek ogólny

- Serwery pośredniczące uaktualniają to pole nagłówka, podczas gdy wysyłane są wiadomości od klienta do serwera i na odwrót. Każdy serwer pośredniczący dodaje do tego pola informację o wykorzystywanym protokole, jego wersji, nazwie hosta, i numerze portu, uzupełniając je komentarzem. Te informacje są bardzo użyteczne w procesie wyszukiwania błędów.

```
Via: protokół host[:port] [ komentarz]
```

Nagłówek żądania

- Określa akceptowane przez klienta typy MIME danych, na przykład Accept: text/*, image/gif Opcjonalna wartość „ranga” reprezentuje ważność poszczególnych parametrów w skali od 0 do 1.

```
Accept: typ/podtyp [q=ranga]
```

- Określa preferowane przez klienta alfabety. Można podać kilka alfabetów. W przypadku, gdy żaden nie został podany ustawiany jest domyślnie US-ASCII, ISO-8859-1.

```
Accept-Charset: alfabet [q=ranga]
```

Nagłówek żądania

- Służy do określenia przez klienta preferowaną wersje językową dokumentów. Jeśli serwer posiada kilka wersji językowych dokumentu to prześle go klientowi w preferowanym przez niego języku. Istnieje możliwość zapisania wielu języków jednocześnie oddzielając je przecinkami. Języki zapisywane są za pomocą dwuliterowych skrótów: en, pl, fr ....

```
Accept-Language: język [q=ranga]
```

- To pole zawiera nazwę cookie związanego z danym adresem URL oraz jego wartość.

```
Cookie: nazwa=wartość
```

Protokoły – HTTP

Nagłówek żądania

- Pole to umożliwia klientowi uzyskanie pozwolenia na dostęp do danych wymagających autoryzacji.

```
Authorization: schemat dane-uwierzytelniające
```

- Pole to zawiera nazwę i opcjonalny numer portu serwera, z którym łączy się klient – domyślnie jest to 80.

```
Host: nazwa hosta [:port]
```

- To pole umożliwia klientowi przekazanie informacji do serwera pośredniczącego w przypadku żądania przez niego autoryzacji.

```
Proxy-Authorization: dane-uwierzytelniające
```

Protokoły – HTTP

Nagłówek żądania

- Pole to pozwala określić zakres lub zakresy treści dokumentu, które mają zostać pobrane, np.: Range: bytes=1024-2047, 4096- Bajty liczone są od 0.

```
Range: bytes=m-n
```

- Zawiera adres URL dokumentu zawierającego odwołanie do adresu żądanego dokumentu.

```
Referer: url
```

- Pole to zawiera informację o programie klienta.

```
User-Agent: nazwa
```

Nagłówek odpowiedzi

- Pole to informuje o możliwości przekazywania jedynie fragmentów treści dokumentu dostępnego pod danym adresem URL.

```
Accept-Ranges: bytes | none
```

- Pole to określa „wiek” dokumentu w sekundach.

```
Age: sekundy
```

- W tym polu przekazywany jest znacznik zawartości, który jest niepowtarzalnym identyfikatorem przypisanym danemu zasobowi, identyfikujący żądany zasób serwera.

```
ETag: znacznik-zawartości
```

- Określa nowe położenie dokumentu.

```
Location: url
```

Nagłówek odpowiedzi

- Pole to zawiera informacje o sposobie i parametrach uwierzytelniania dla serwera pośredniczącego obsługujące dane połączenie i odwołanie do bieżącego adresu URL.

```
Proxy-Authenticate: schemat obszar-autoryzacji
```

- Informuje klienta, kiedy serwer będzie mógł przetworzyć żądanie i stosować je w odpowiedziach.

```
Retry-After: data | sekundy
```

- Zawiera informacje o nazwie i wersji serwera.

```
Server: łańcuch
```

Nagłówek odpowiedzi

- Definiuje nazwę i wartość cookie związanego z danym adresem URL i przeznaczony jest wyłącznie dla przeglądarek obsługujących mechanizm cookies.

```
Set-Cookie: nazwa=wartość opcje
```

- Pole to informuje, że żądany dokument dostępny jest w kilku wersjach, w związku z tym odpowiedź może być różna w zależności od zawartości pól nagłówka żądania..

```
Vary: pole-nagłówka
```

Nagłówek odpowiedzi

- Oznacza żądanie uwierzytelniania. Zawiera ono informacje o sposobie uwierzytelniania oraz nazwie obszaru autoryzacji, przekazywane klientowi żądającemu dostępu do danego adresu URL.

```
WWW-Authenticate: schemat obszar-autoryzacji
```

Protokoły – HTTP

Nagłówek zawartości

- Pole zawiera listę metod dozwolonych dla danego adresu URL.

```
Allow: metoda HTTP
```

- Zawiera informację o języku lub językach używanych w treści dokumentu, która jest przesyłana. Języki są identyfikowane za pomocą dwuliterowych kodów. W miarę możliwości serwer powinien wysłać dokument w wersji językowej zgodnej z preferencjami klienta, które określone są w odpowiednim polu.

```
Content-Language: język
```

- Zawiera informację o rozmiarze treści przesyłanego dokumentu w bajtach.

```
Content-Length: rozmiar
```

Protokoły – HTTP

Nagłówek zawartości

- Zawiera adres URL treści dokumentu w przypadku, gdy posiada on wiele wariantów dostępnych pod różnymi adresami.

```
Content-Location: url
```

- Zawiera skrót MD5 przesyłanej treści, umożliwiający sprawdzenie integralności wiadomości po jej odebraniu.

```
Content-MD5: skrót-md5
```

- Określa położenie danego fragmentu w całości przekazywanego dokumentu, a także całkowity rozmiar treści dokumentu.

```
Content-Range: bytes n-m/k
```

- Opisuje typ MIME treści dokumentu.

```
Content-Type: typ-MIME
```

Protokoły – HTTP

Nagłówek zawartości

- Określa termin możliwej zmiany lub wygaśnięcia ważności dokumentu. Po upływie określonego tutaj czasu dokument może, choć nie musi, zostać zmodyfikowany lub usunięty.

```
Expires: data
```

- Określa datę ostatniej modyfikacji dokumentu znajdującego się pod danym adresem URL.

```
Last-Modified: data
```

Protokoły – HTTP

Przykład 1 – Sprawdzenie jakie metody wspiera serwer

- Żądanie strony

```
OPTIONS /a1.html HTTP/1.1
Host: 10.0.1.2
¶
```

- Odpowiedź serwera

```
HTTP/1.1 200 OK
Date: Wed, 28 Feb 2007 13:05:26 GMT
Server: Apache/2.0.59 (Win32) PHP/5.1.6
Vary: accept-language,accept-charset
Allow: GET,HEAD,POST,OPTIONS,TRACE
Content-Length: 0
Content-Type: text/html
Content-Language: en
```

Protokoły – HTTP

Przykład 2 – Pobranie nieistniejącej strony

- Żądanie strony

```
GET /a1.html HTTP/1.1
Host: 10.0.1.2
¶
```

- Odpowiedź serwera

```
HTTP/1.1 404 Not Found
Date: Wed, 28 Feb 2007 12:59:58 GMT
Server: Apache/2.0.59 (Win32) PHP/5.1.6
Content-Length: 288
Content-Type: text/html; charset=iso-8859-1
¶
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head><title>404 Not Found</title></head>
<body>
<h1>Not Found</h1>
<p>The requested URL /a1.html was not found on this server.</p><hr>
<address>Apache/2.0.59 (Win32) PHP/5.1.6 Server at 10.0.1.2 Port 80 </address>
</body>
</html>
```

Protokoły – HTTP

Przykład 3 – Pobranie strony

- Żądanie strony

```
GET /a1.html HTTP/1.1
Host: 10.0.1.2
¶
```

- Odpowiedź serwera

```
HTTP/1.1 200 OK
Date: Wed, 28 Feb 2007 13:09:20 GMT
Server: Apache/2.0.59 (Win32) PHP/5.1.6
Last-Modified: Wed, 28 Feb 2007 13:09:01 GMT
ETag: "8f2e-32-1402bf45"
Accept-Ranges: bytes
Content-Length: 50
Content-Type: text/html
¶
<html>
<body>
          STRONA TESTOWA
</body>>
</html>
```

Przykład 4 – Strona chroniona hasłem

- Konfiguracja serwera

```
AuthType Basic
AuthName "Dla studentow"
AuthUserFile "C:\a1"
require valid-user

Użytkownik              jarek
Hasło                   qwe123
```

Przykład 4 – Strona na hasło

- Żądanie strony

```
GET /a1.html HTTP/1.1
Host: 10.0.1.2
¶
```

- Odpowiedź serwera

```
HTTP/1.1 401 Authorization Required
Date: Wed, 28 Feb 2007 13:25:31 GMT
Server: Apache/2.0.59 (Win32) PHP/5.1.6
WWW-Authenticate: Basic realm="Dla studentow"
Content-Length: 484
Content-Type: text/html; charset=iso-8859-1
¶
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
….
        This server could not verify that you are authorized to access the document requested.
Either you supplied the wrong credentials (e.g., bad password), or your browser doesn't
understand how to supply
the credentials required
```

Protokoły – HTTP

Przykład 4 – Strona na hasło

- Żądanie strony

```
GET /a1.html HTTP/1.1
Host: 10.0.1.2
AUTHORIZATION: Basic jarek:qwe123
¶
```

- Odpowiedź serwera

```
HTTP/1.1 401 Authorization Required
Date: Wed, 28 Feb 2007 13:25:31 GMT
Server: Apache/2.0.59 (Win32) PHP/5.1.6
WWW-Authenticate: Basic realm="Dla studentow"
Content-Length: 484
Content-Type: text/html; charset=iso-8859-1
¶
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
This server could not verify that you are authorized to access the document requested.
Either you supplied the wrong credentials (e.g., bad password), or your browser doesn't
understand how to supply
the credentials required
```

Protokoły – HTTP

Przykład 4 – Strona na hasło

jarek:qwe123
(zakodowane Base64)



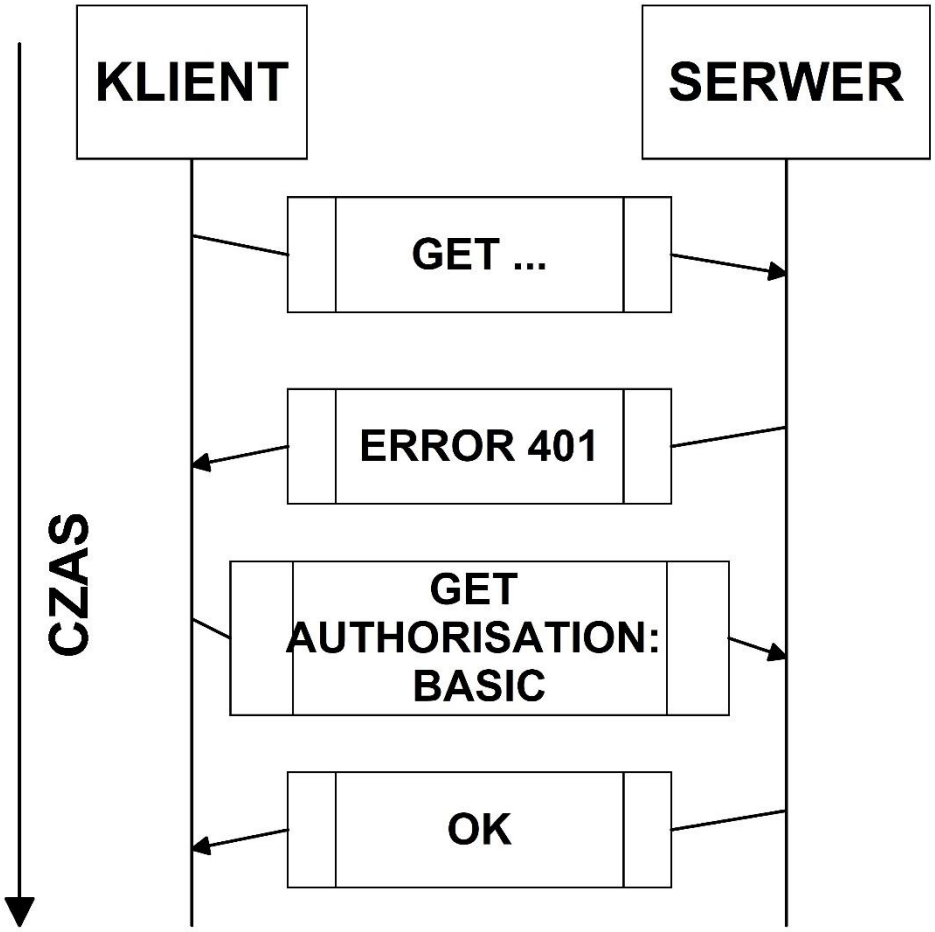
- Żądanie strony

```
GET /a1.html HTTP/1.1
Host: 10.0.1.2
AUTHORIZATION: Basic amFyZWs6cXdlMTIz
¶
```

- Odpowiedź serwera

```
HTTP/1.1 200 OK
Date: Wed, 28 Feb 2007 13:50:05 GMT
Server: Apache/2.0.59 (Win32) PHP/5.1.6
Last-Modified: Wed, 28 Feb 2007 13:09:01 GMT
ETag: "8f2e-32-1402bf45"
Accept-Ranges: bytes
Content-Length: 50
Content-Type: text/html
¶
<html><body> STRONA TESTOWA</body></html>
```

Przykład 4 – Strona na hasło

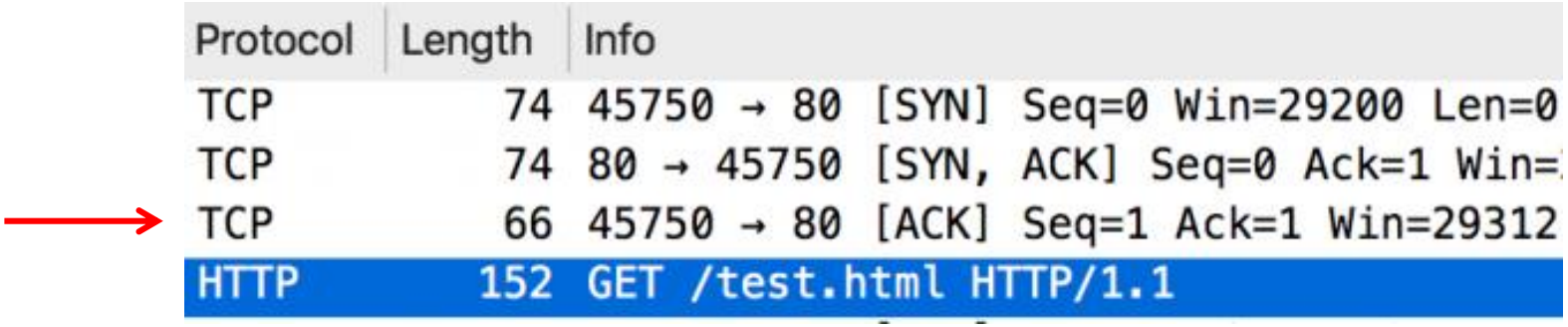


PROTOKÓŁ HTTP 2.0

Protokoły – HTTP

Nawiązanie połączenia

- Zestawienie połączenia
  - ✓ http 1.1



Protokoły – HTTP

Nawiązanie połączenia

- Zestawienie połączenia
  - ✓ http 2.0

Strumień zerowy służy do przesyłania parametrów kontrolnych całego połączenia

Protocol	Length	Info
TCP	74	45746 → 80 [SYN] Seq=0 Win=29200 Len=0
TCP	74	80 → 45746 [SYN, ACK] Seq=0 Ack=1 Win=
TCP	66	45746 → 80 [ACK] Seq=1 Ack=1 Win=29312
HTTP2	90	Magic
TCP	66	80 → 45746 [ACK] Seq=1 Ack=25 Win=2905
HTTP2	87	SETTINGS[0]
TCP	66	80 → 45746 [ACK] Seq=1 Ack=46 Win=2905
HTTP2	79	WINDOW_UPDATE[0]
TCP	66	80 → 45746 [ACK] Seq=1 Ack=59 Win=2905
HTTP2	113	HEADERS[1]: GET /test.html

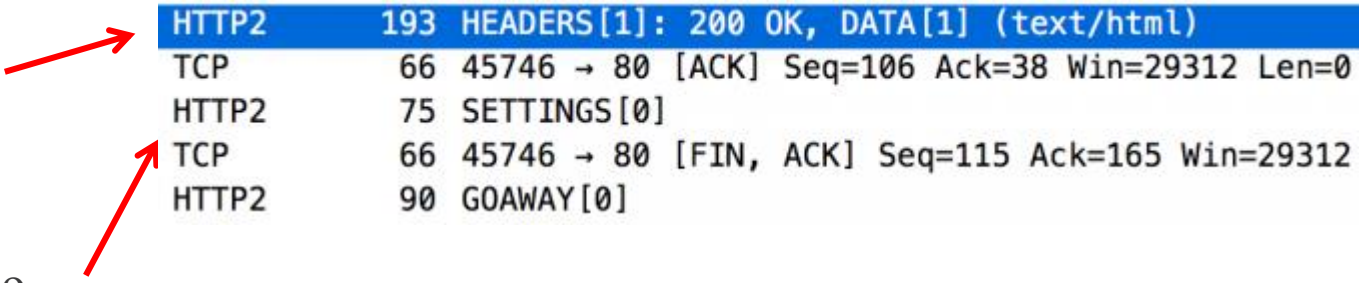
Z kolei strumień zapytania został stworzony ramką HEADERS [1].

Protokoły – HTTP

Nawiązanie połączenia

- Zestawienie połączenia
  - ✓ http 2.0

Odpowiedz na żądanie HEADERS [1].



Strumień zerowy służy do przesyłania parametrów kontrolnych całego połączenia

PROTOKÓŁ FTP

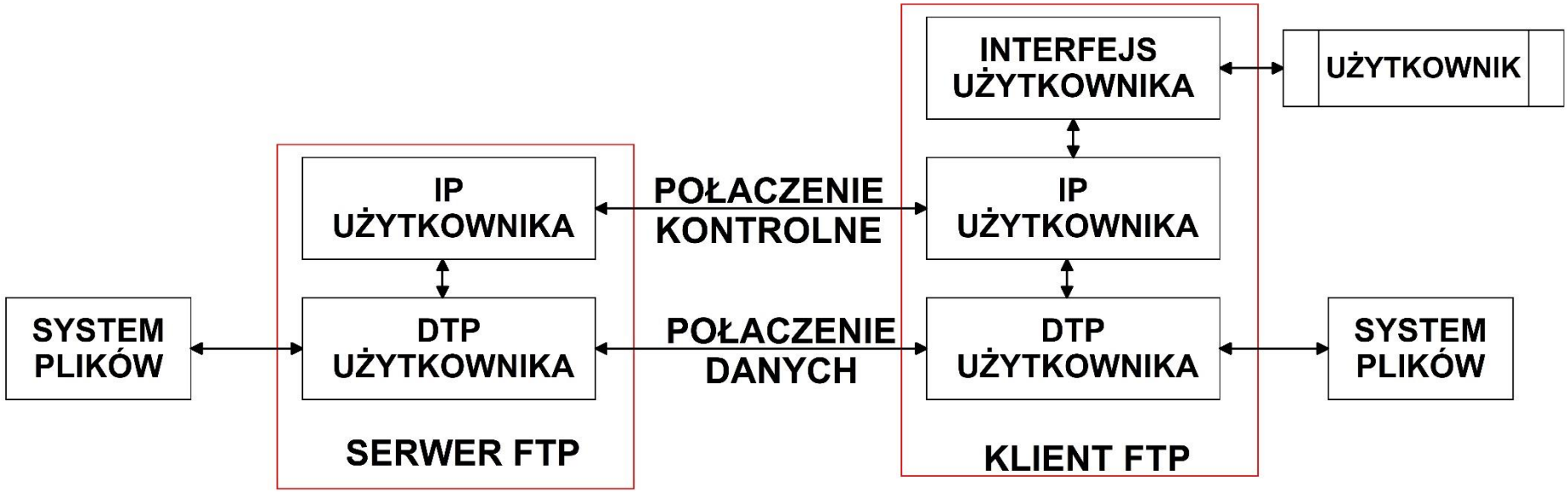
FTP jest najpopularniejszą usługą służącą do przesyłania plików poprzez sieć. Działa ona na zasadzie klient serwer to znaczy na jednej maszynie znajduje się odpowiednie oprogramowanie służące do udostępniania zasobów i do niego korzystając z odpowiedniego oprogramowania podłączają się użytkownicy.

Protokoły – FTP

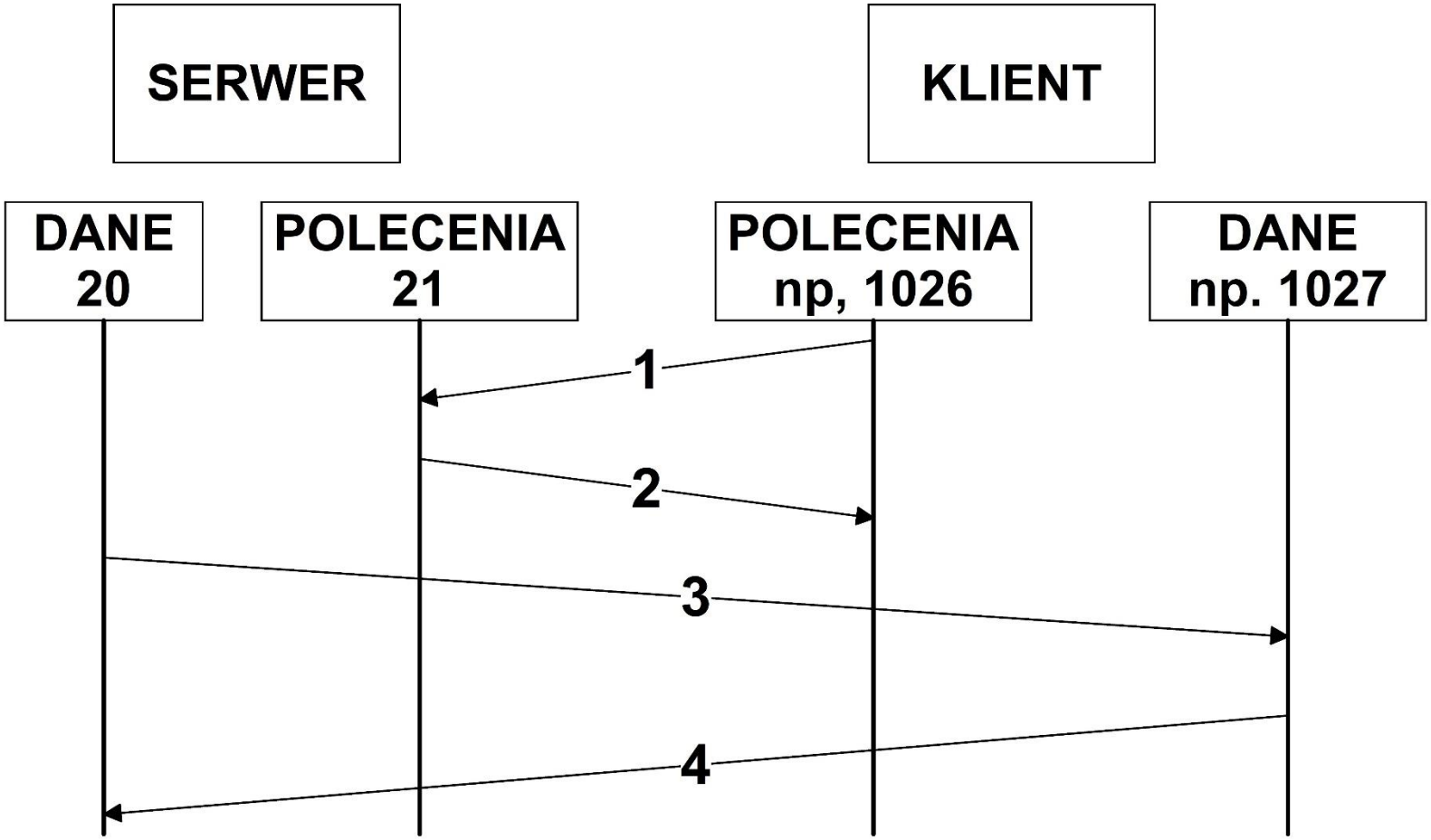
Zasada działania – Usługa FTP wykorzystuje do przenoszenia danych w sieci między komputerami dwa procesy

- Interpretator Protokół PI (Protocal Interpreter) wykorzystywany do przesyłania poleceń między komputerem klienta a serwerem. Inicjuje on usługę FTP i zarządza usługą DTP od strony klienta.
- Proces przesyłania danych DTP (Data Transfer Proces) zapewniający faktyczną transmisję danych między komputerem klienta a serwerem.

Zasada działania

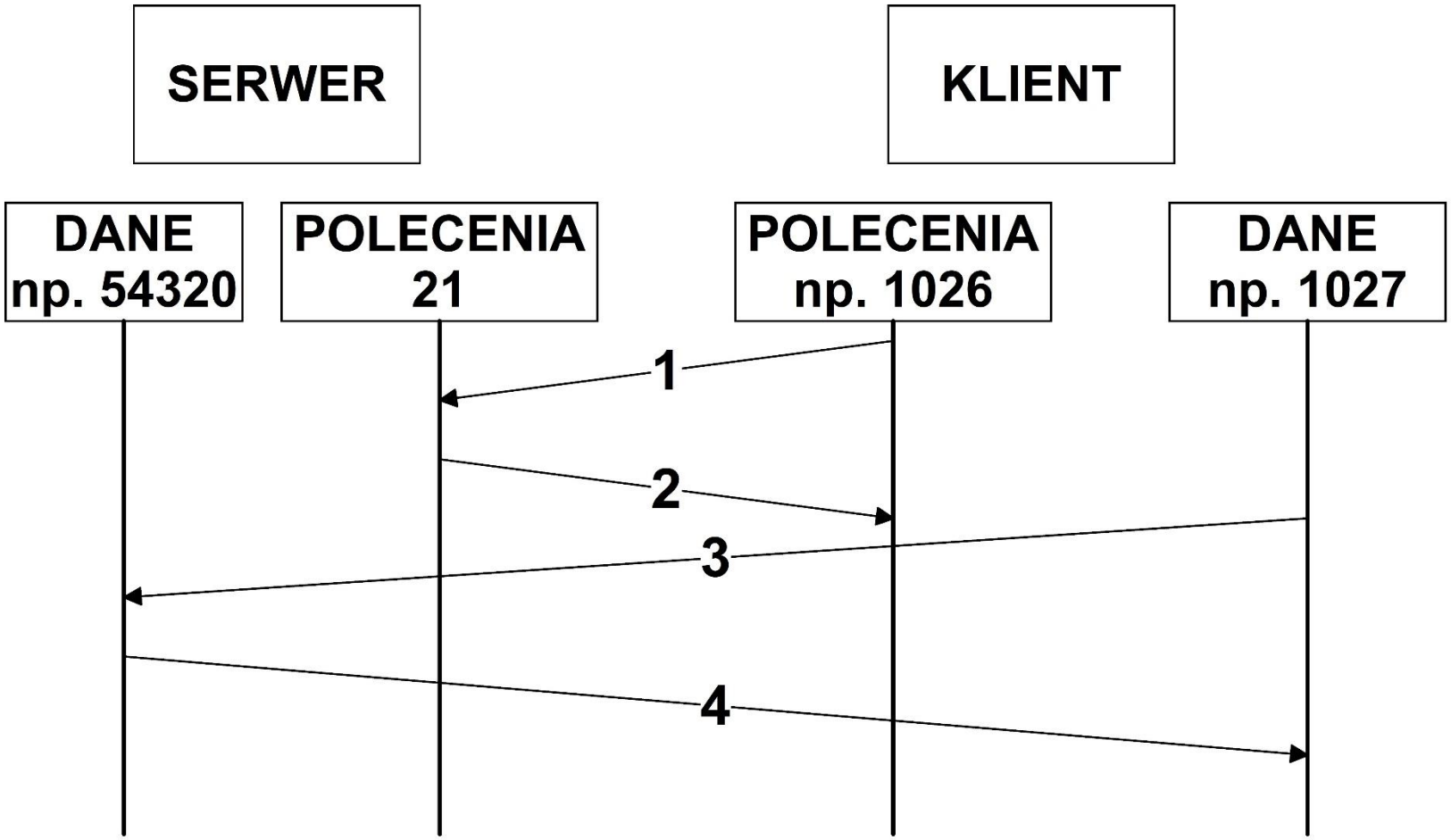


Zasada działania – tryb aktywny



Protokoły – FTP

Zasada działania – tryb pasywny



Protokoły – FTP

Polecenia kontroli dostępu

- **USER użytkownik** – Argumentem tej komendy jest ciąg znaków identyfikujących logującego się użytkownika.
- **PASS hasło** – Argumentem tej komendy jest ciąg znaków będących hasłem logującego się użytkownika.
- **ACCT konto** – Argumentem tej komendy jest ciąg znaków określający konto użytkownika. Komenda ta nie jest bezpośrednio związana z komendą USER, lecz niektóre serwery jej wymagają w celu określenia dostępu.

Protokoły – FTP

Polecenia kontroli dostępu

- **REIN** – Komenda ta rozłącza użytkownika, czyszcząc zajęte przez niego porty wejścia/wyjścia oraz pozwalają trwającemu już transferowy na jego dokończenie.
- **QUIT** – Komenda ta rozłącza użytkownika i jeżeli nie jest kopiowany żaden plik, serwer zamyka połączenie. Jeżeli transfer pliku jest w toku, połączenie pozostaje otwarte do momentu otrzymania odpowiedzi o przerwaniu transferu, a następnie serwer zamyka połączenie.

Protokoły – FTP

Polecenia sterujące przesyłaniem

- **PORT port** – Argumentem jest numer portu serwera dla data portu użytego przy tworzeniu kanału.
- **PASV** – Komenda ta prosi serwer, aby ten słuchał na data port, (który nie jest standardowym data portem) i czekał na połączenie a nie sam inicjował połączenie.

```
227 Entering Passive Mode (10,0,0,4,9,149).
```

Ip serwera: 10.0.0.4

Port: 2453

Protokoły – FTP

Polecenia definiujące transfer plików

- **RETR plik** – Komenda ta powoduje, iż serwer transferuje kopie pliku, sprecyzowanego w ścieżce dostępu, do innego serwera lub użytkownika na drugim końcu data connection. Status i zawartość pliku, na serwerze powinna zostać niezmieniona.
- **STOR plik** – Ta komenda sprawia, że serwer zaakceptuje dane przesłane do niego za pomocą data connection i zapisze je na nim jako plik. Jeżeli plik o tej nazwie już na serwerze istnieje to użytkownik mając odpowiednie prawa może nadpisać istniejący plik. Jeżeli plik taki nie istnieje to tworzony jest nowy.

Protokoły – FTP

Polecenia definiujące transfer plików

- **STOU plik** – Komenda ta zachowuje się podobnie do STOR z tą różnicą, że jeżeli na serwerze w danym folderze istnieje już plik o tej samej nazwie, z jaką chcemy zapisać nowy plik generowana jest nowa nazwa pod jaką zostaje ten plik zapisany. Odpowiedź zawiera 250 Transfer started wraz z wygenerowaną nazwą.
- **APPE plik** – Ta komenda sprawia, że serwer zaakceptuje dane przesłane do niego za pomocą data connection i zapisze je na nim jako plik. Jeżeli plik sprecyzowany w ścieżce dostępu istnieje na serwerze, wtedy dane zostaną dodane do istniejących. W przeciwnym wypadku plik sprecyzowany w ścieżce dostępu zostanie stworzony od nowa.

Protokoły – FTP

Polecenia definiujące transfer plików

- **ALLO przydziel** – Komenda ta może być wymagana przez niektóre serwery do rezerwacji wystarczającej ilości miejsca do zapisania w całości transferowanego pliku. Argumentem tej komendy jest rozmiar pliku, który ma być transferowany.
- **REST miejsce plik** – Pole argumentu reprezentuje miejsce, od którego ma być wznowiony transfer wskazanego pliku. Komenda ta nie powoduje transferu całego pliku, ale skok do miejsca, w którym transfer został przerwany.
- **ABOR** – Komenda ta mówi serwerowi, aby ten przerwał poprzednią komendę usług FTP i każdy związany z nią transfer. Zamykany jest data connection, ale control connection nie, przez co można wykonywać inne operacje.

Protokoły – FTP

Polecenia definiujące transfer plików

- **DELE plik** – Komenda ta powoduje, że plik określony ścieżką dostępu zostaje usunięty z serwera.
- **RNFR plik** – Komenda określa starą ścieżkę dostępu do pliku, która ma być zmieniona na nową. Po tej komendzie musi być wprowadzona komenda RNTO określająca nową ścieżkę dostępu.
- **RNTO plik** – Komenda, która musi wystąpić zaraz po RNFR i która określa nową ścieżkę dostępu do pliku.
- **RMD katalog** – Komenda powoduje usunięcie folderu określonego ścieżką dostępu.
- **MKD katalog** – Komenda z kolei powoduje utworzenie folderu w ścieżce dostępu.

Protokoły – FTP

Polecenia definiujące transfer plików

- **PWD** – Wyświetla nazwę folderu, w którym aktualnie się znajdujemy.
- **LIST** – Powoduje, że serwer wysyła zawartość folderu określoną ścieżką dostępu. Dane zawierające zawartość folderu przesyłane są przez data connection jako typ ASCII lub EBCDIC
- **NLST** – Komenda przesyła tylko i wyłącznie ścieżki i nazwy plików bez dodatkowych informacji.
- **SYST** – Komenda ta pozwala na sprawdzenie, na jakim systemie operacyjnym jest wystartowany serwer.

Protokoły – FTP

Polecenia definiujące transfer plików

- **STAT** – Wyświetla status połączenia przesyłając tę informacje przez control connection w formie odpowiedzi. Komenda ta może być przesłana do serwera w trakcie transferu pliku lub podczas przerwy miedzy transferami. Użyta z parametrem w postaci ścieżki dostępu zwraca jej zawartość nie przez data connection lecz przez control connection zachowując się podobnie jak komenda LIST tylko używając innego kanału.
- **HELP** – Powiadamia serwer aby ten wysłał pełną informacje o swoim stanie i listę obsługiwanych komend. Specyfikacja zaleca również, aby komenda HELP mogła być dostępna jeszcze przed logowaniem, czyli przed komenda USER.
- **NOOP** – Komenda służy do podtrzymywania połączenia z serwerem mimo bezczynności użytkownika.

Protokoły – FTP

Klient FTP

- Każdy system operacyjny zawiera prostego klienta ftp dostarczanego wraz z nim.
- Wiele tego typu programów jest integralna częścią innych np.:
  - ✓ Total Commander
  - ✓ Przeglądarka IE zawiera klienta ftp.
- Jeśli zależy nam na programie z dużymi możliwościami, należy wybrać dedykowany do tego program.

Protokoły – FTP

Klient FTP

- Każdy z tych klientów zawiera zestaw poleceń (niestety nieco różniących się od protokołowych) umożliwiających komunikacji za pomocą protokołu FTP. Polecenia te można podzielić na:
  - ✓ Polecenia sterujące dostępem.
  - ✓ Polecenia sterujące przesyłaniem.
  - ✓ Polecenia przesyłania plików.
  - ✓ Polecenia zarządzania plikami i katalogami.
  - ✓ Polecenia pomocy i kontroli stanu sesji

Protokoły – FTP

Klient FTP – polecenia sterujące dostępem

- **OPEN [komputer]** – Ustanawia sesję z usługą FTP stacji o nazwie komputer.
- **USER [nazwa]** – Określa użytkownika korzystającego z poleceń FTP.
- **PASS [hasło]** – Polecenie wprowadzane bezpośrednio po USER. Przekazuje ono hasło użytkownika do serwera FTP
- **QUIT** – Kończy sesję łączącą stacje klienta i serwera

Protokoły – FTP

Klient FTP – polecenia sterujące przesyłaniem

- **PORT ##** – Polecenie to pozwala wybrać numer portu po stronie klienta. Parametr ## powinien zawierać 32-bitowy adres IP stacji, z którą nawiązane będzie połączenie DTP oraz l6-bitowy numer portu.
- **PASV** – Polecenie Passive (Pasywny) zmienia zachowanie serwera przy ustanawianiu sesji danych. Zamiast inicjowania połączenia, serwer będzie jedynie monitorował port danych i oczekiwał na ustanowienie sesji przez klienta.
- **TYPE** – Polecenie Representation Type (Typ reprezentacji) określa format reprezentacji danych serwera: ASCII, EBCDIC lub Image.

Protokoły – FTP

Klient FTP – polecenia przesyłania plików

- **HASH** – Przełącza drukowanie znaków # po przesłaniu każdego bloku (1024 bajtów) danych, pozwala to optycznie śledzić tempo przesyłania informacji.
- **ASCII** – Włącza tryb transmisji ASCII, dla plików tekstowych. Jest on używany domyślnie.
- **BELL** – Po zakończeniu transferu wydany będzie sygnał dźwiękowy.
- **BINARY** – Włącza tryb binarny transmisji, który powinien być stosowany do przesyłania wszystkich plików innych niż pliki tekstowe.

Protokoły – FTP

Klient FTP – polecenia przesyłania plików

- **RECV plikserwer [plikklient]** – Kopiuje plik plikserwer z serwera do pliku lokalnego plikklient, Jeżeli nazwa pliku lokalnego nie zostanie podana, plik zostanie skopiowany z zachowaniem nazwy oryginalnej.
- **SEND plikklient [plikserwer]** – Kopiuje lokalny plik plikklient do pliku plikserwer na serwerze. Jeżeli nazwa pliku na serwerze nie zostanie podana, plik zostanie skopiowany z zachowaniem nazwy lokalnej.
- **GET plikserwer [plikklient]** – Funkcjonuje tak samo jak polecenie recv.
- **PUT plikklient [plikserwer]** – Funkcjonuje tak samo jak polecenie send.

Protokoły – FTP

Klient FTP – polecenia przesyłania plików

- **MGET [plikserwer]** – Multiple Get (Wielokrotne Get) pozwala na kopiowanie większej ilości plików (symbole wieloznaczne).
- **MPUT [plikklient]** – Multipłe Put (Wielokrotne PUT) pozwala na kopiowanie większej ilości plików.
- **PROMPT** – Włącza lub wyłącza wyświetlanie komunikatów z żądaniem potwierdzenia kopiowania po wprowadzeniu poleceń mget i mput.
- **DELETE plikserwer** – Usuwa plik o nazwie plikserwer z serwera FTP.
- **MDELETE plikserwer [...]** – Usuwa wszystkie pliki odpowiadające opartemu na symbolach wieloznacznych wzorcowi plikserwer.

Protokoły – FTP

Klient FTP – Polecenia zarządzania plikami i katalogami

- **LCD –** Polecenie Local Change Directory (Zmień katalog lokalny) zmienia lokalny katalog bieżący.
- **CD [katalog] –** Polecenie Change Directory (Zmień katalog) zmienia katalog bieżący na serwerze.
- **CDUP –** Polecenie Change Directory Up (Zmień katalog „do góry") zmienia katalog bieżący na serwerze na katalog o poziom wyżej.
- **MKDIR katalog** – Polecenie Make Directory (Utwórz katalog) zakłada nowy katalog na serwerze FTP.
- **RMDIR katalog** – Polecenie Remove Directory (Usuń katalog) usuwa katalog na serwerze.
- **DIR** – Wyświetla zawartość bieżącego katalogu serwera ze szczegółami. Może ona zostać zapisana do pliku.

Protokoły – FTP

Klient FTP – Polecenia zarządzania plikami i katalogami

- **LS** – Polecenie wyświetla zawartość bieżącego katalogu serwera w formie skróconej.
- **PWD** – Wyświetla nazwę bieżącego katalogu serwera.
- **RENAME Naz1 Naz2** – Zmienia nazwę pliku na serwerze z Naz1 na Naz2.

Protokoły – FTP

Protokół FTP – kody odpowiedzi

- Na każde nasze polecenie serwer odpowiada specjalnym trzyznakowym kodem:

```
XYZ   Opis słowny
```

- Pierwsza cyfra kodu oznacza:

1. Pozytywna odpowiedź wstępna. Żądana akcja została podjęta.
2. Pozytywna odpowiedź końcowa.
3. Pozytywna odpowiedź pośrednia. Serwer zaakceptował polecenie, ale oczekiwane są dalsze dane.
4. Negatywna odpowiedź tymczasowa.
5. Negatywna odpowiedź trwała.

Protokoły – FTP

Protokół FTP – kody odpowiedzi

- Druga cyfra kodu oznacza:
  1. Składnia. Błąd wynika z niewłaściwej składni albo też wydania polecenia poprawnego, ale w nieodpowiednim momencie.
  2. Dane. Kod wykorzystywany w odpowiedziach na polecenia pomocy i kontroli stanu sesji.
  3. Połączenia. Kod używany w odpowiedziach odnoszących się do stanu połączenia danych.
  4. Nieokreślony. Kod jeszcze nieużywany.
  5. System plików. Informacja o stanie systemu plików serwera lub klienta FTP.
- Trzecia cyfra jeszcze dokładnie precyzuje odpowiedź serwera

Protokoły – FTP

Przykładowy przebieg sesji FTP

```
open 10.0.1.2 21
   220 Plase enter your name
user sk_1
   331 user name ok
pass sk_1
   230 use login
pasv
   227 enter passive mode
ls
   200 port command successful
   150 open transfer
autoexec.bat
   226 transfer successful
quit
```

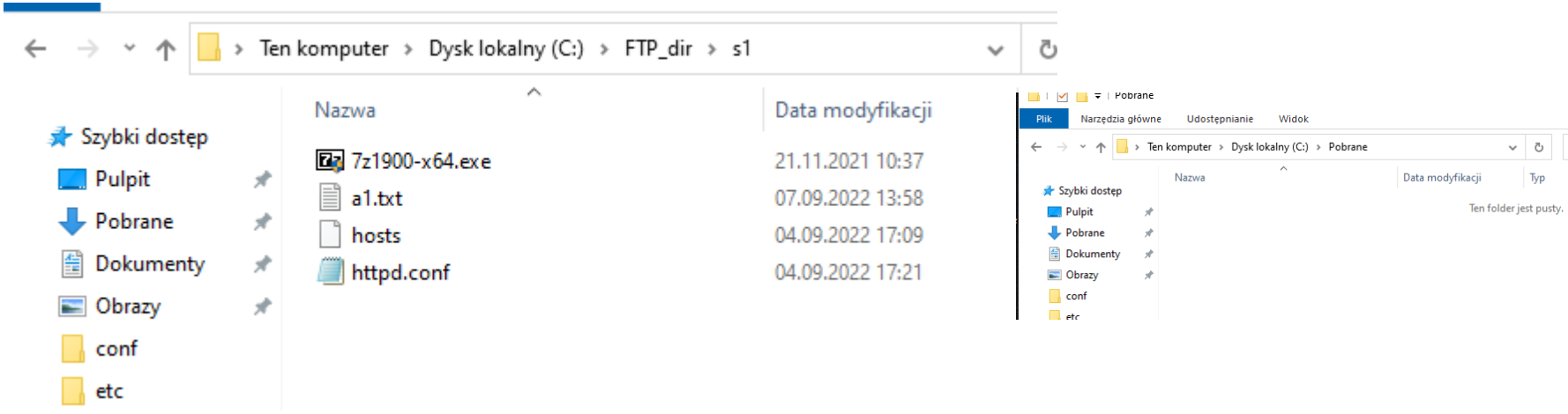
Protokoły – FTP

Klient FTP

- W wierszu poleceń należy wydać polecenie ftp. Możliwe parametry
  - **v** Pominiecie odpowiedzi zdalnego serwera.
  - **n** Eliminacja automatycznego logowania
  - **i** Wyłącza komunikaty przy ściąganiu wielu plików.
  - **s:plik** Czyta komendy z pliku.
  - **A** Załoguj się jako użytkownik anonimowy.
  - **p port** numer portu
  - **u użytkownik hasło.** Przeprowadź autoryzację jako podany użytkownik

Klient FTP - Przykład

- Napisać plik wsadowy, który z serwera o nazwie podanej jako pierwszy parametr wywołania do katalogu podanego jako drugi parametr wywołania ściągnie z konta ***student1*** o haśle ***student1*** wszystkie pliki, jeśli pliki już istnieją to wykonać ich kopie w katalogi data\godzina (data w układzie miesiąc-dzień). Wszystkie operacje tymczasowe wykonywane w katalogu tmp



Klient FTP – przykład rozwiązanie

Plik Edycja Format Widok Pomoc

```
@ECHO OFF
REM - PARAMETER PIERWZY NAZWA SERWERA
REM - PARAMETER DRUGI NAZWA KATALOG DOCELOWEGO
ECHO PRZEGRYWAM DANE Z SERWERA   : %1
ECHO DO KATALOGU DOCELOWEGO      : %2
set godz=%time:~0,5%
set go=%time:~0,2%
set mi=%time:~3,2%
set godz1=%go%%m1%
set mies=%DATE:~0,2%
set dzi=%DATE:~3,2%
set dat1=k%dzi%%mies%%go%%mi%
ECHO DZISIAJ JEST %DATE:~3% GODZINA %godz%
ECHO KATALOG TYMCZASOWY TMP
if EXIST TMP DEL TMP/*.*
if EXIST TMP RMDIR TMP
MKDIR TMP
IF NOT EXIST %2 MKDIR %2
cd TMP
```

```
C:\pobrane>a1.bat 10.0.0.41 kat1
PRZEGRYWAM DANE Z SERWERA       : 10.0.0.41
DO KATALOGU DOCELOWEGO          : kat1
DZISIAJ JEST 10.2023 GODZINA 11:07
KATALOG TYMCZASOWY TMP
Katalog nie jest pusty.
```

Protokoły – FTP

Klient FTP – przykład rozwiązanie

```
ftp -i -s:..\polecenia.txt %1
echo PLIKI SCIAGNIETE
for %%a in (*.*) do call ..\a2.bat %2 %%a %dat1%
move *.* ..\%2\
cd ..
```

Plik Edycja Fc

```
student1
student1
binary
mget * .
bye
```

```
230 Login successful.
ftp> binary
200 Type set to I
ftp> mget * .
Couldn't open the file or directory
200 Type set to I
200 PORT command successful.
150 Starting data transfer.
226 Operation successful
ftp: 1447178 bytes received in 0.02Seconds 85128.12Kbytes/sec.
200 PORT command successful.
150 Starting data transfer.
226 Operation successful
ftp: 12 bytes received in 0.00Seconds 12000.00Kbytes/sec.
200 PORT command successful.
150 Starting data transfer.
226 Operation successful
ftp: 824 bytes received in 0.00Seconds 824000.00Kbytes/sec.
200 PORT command successful.
150 Starting data transfer.
226 Operation successful
ftp: 20940 bytes received in 0.00Seconds 20940000.00Kbytes/sec.
ftp> bye
221 Goodbye.
PLIKI SCIAGNIETE
```

Plik Edycja Format Widok Pomoc

```
if not exist ..\%1\%2   goto  KONIEC
if not exist ..\%1\%3   mkdir ..\%1\%3
 move %2 ..\%1\%3\
: KONIEC
```

Nazwa	Data modyfikacji
7z1900-x64.exe	26.10.2023 11:25
a1.txt	26.10.2023 11:25
hosts	26.10.2023 11:25
httpd.conf	26.10.2023 11:25

k10261130	26.10.2023 11:30
7z1900-x64.exe	26.10.2023 11:25
a1.txt	26.10.2023 11:25
hosts	26.10.2023 11:25
httpd.conf	26.10.2023 11:25

Protokoły – FTP

FTP Serwery – Istnieje wiele różnych programów umożliwiających świadczenie usługi serwera ftp od wbudowanych w system operacyjny poprzez systemy freeware, po komercyjne za kilka czy kilkaset dolarów.

- Cech serwera
  - ✓ Zakładanie dowolnej ilość bez szelowych kont.
  - ✓ Umożliwiać dla każdego konta:
    - ✓ Ustalenie limitu ilości połączeń.
    - ✓ Ustalenie zakresów adresów ip, z których można się połączyć.
    - ✓ Ustalenie zakresu czasu połączeń.
    - ✓ Ustalenie limitu prędkości transferu danych.
  - ✓ Pełne logowanie wszystkich sesji użytkowników.
  - ✓ Być darmowy
- Na rynku jest wiele produktów, które spełniają podane warunki.

Poczta elektroniczna

Poczta elektroniczna jest usługą służącą do wysyłania informacji innym użytkownikom sieci w czasie ich nieobecności.

Protokoły – SMTP, POP3

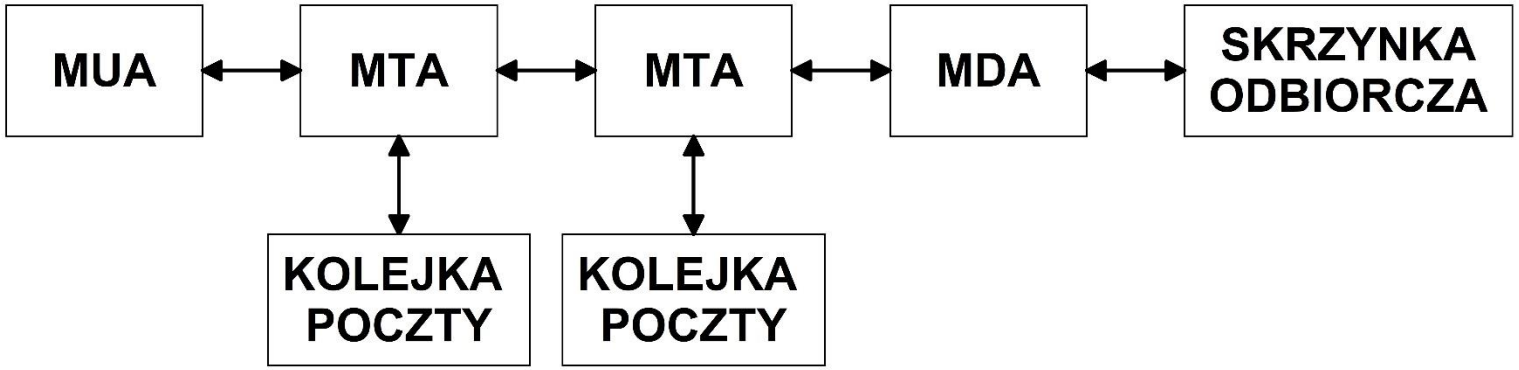
Zasada działania

- Działa zarówno w obszarze sieci lokalnej jak i sieci WAN.
- Pracuje w architekturze klient serwer to znaczy:
  - ✓ Użytkownik chcąc wysłać pocztę elektroniczną do innego użytkownika sieci (oczywiście musi znać jego adres i sam posiadać konto pocztowe) łączy się ze swoim serwerem i wysyła do niego pocztę.
  - ✓ Jego serwer przesyła ją do serwera, na którym posiada konto odbiorca.
  - ✓ Tam ona oczekuje aż użytkownik podłączy się do serwera i ją odbierze.

Protokoły – SMTP, POP3

Zasada działania

- Do przekazywania poczty używane są trzy rodzaje programów. W skrócie oznacza się je jako:
  - ✓ MUA (Mail User Agent),
  - ✓ MTA (Mail Transfer Agent)
  - ✓ MDA (Mail Delivery Agent).
- Odpowiadają one za poszczególne etapy wędrówki wiadomości pocztowej w Sieci



Protokoły – SMTP, POP3

Zasada działania – adres pocztowy

- Prosty adres e-mailowy składa się z
  - ✓ części lokalnej (czyli nazwy skrzynki pocztowej),
  - ✓ znaku "małpy" ("@")
  - ✓ domeny.
- Przykład
  - ✓ bopjw@tu.kielce.pl
  - ✓ j.wikarek@tu.kielce.pl

Protokoły – SMTP, POP3

Zasada działania – wysyłanie poczty

- Do wysyłania poczty używa się protokołu SMTP - Simple Mail Transfer Protocol (specyfikację protokołu SMTP - opisuje norma RFC 821).
- Standardowym portem przypisanym do usługi SMTP jest port 25.
- W SMTP trudno jest wyszczególnić pojęcie serwera i klienta. Tak naprawdę mamy do czynienia z nadawcą (sender) i odbiorcą (receiver).

Protokoły – SMTP, POP3

Zasada działania – Protokół SMTP

❖ **HELO domena**	Przywitanie się z serwerem.
❖ **MAIL FROM: <adres@nadawcy>**	Określenie nadawcy.
❖ **RCPT TO: <adres@odbiorcy>**	Określenie odbiorcy.
❖ **DATA**	Początek transferu danych.
❖ **.**	Koniec danych.
❖ **QUIT**	Zakończenie sesji.
❖ **REST**	Wyczyszczenie sesji.
❖ **ARFY**	Potwierdzenie danych adresata.

Protokoły – SMTP, POP3

Zasada działania – Protokół SMTP polecenia opisane ale niewymagane przez RFC 821

- **SOML SEND OR MAIL** – Pozwala przekazać wiadomość do skrzynki pocztowej lub terminalu użytkownika.
- **SAML SEND AND MAIL** – Pozwala przekazać wiadomość jednocześnie do skrzynki pocztowej i terminalu użytkownika.
- **EXPN** – Pozwala uzyskać w odpowiedzi listę uczestników listy wysyłkowej.
- **HELP** – Żądanie przesłania przez odbiorcę pomocy dla użytkownika.
- **TURN** – Żądanie przyjęcia przez odbiorcę roli stacji wysyłającej.

Protokoły – SMTP, POP3

Zasada działania – Protokół SMTP – kody odpowiedzi.

- Odpowiedz serwera składa się z trzech cyfr i komunikatu słownego

```
XYZ odpowiedz serwera
```

- Pierwsza cyfra:
  - ✓ 2 odpowiedź pozytywna.
  - ✓ 3 odpowiedź częściowo pozytywna, odbiorca oczekuje na kolejne dane.
  - ✓ 4 odpowiedź chwilowo negatywna.
  - ✓ 5 odpowiedź jednoznacznie negatywna.

Protokoły – SMTP, POP3

Zasada działania – Protokół SMTP – kody odpowiedzi.

- Odpowiedz serwera składa się z trzech cyfr i komunikatu słownego

```
XYZ odpowiedz serwera
```

- Druga cyfra:
  - ✓ x0x błąd składni.
  - ✓ x1x odpowiedź informacyjna.
  - ✓ x2x odpowiedź dotycząca połączenia lub sesji.
  - ✓ x3x nie określono
  - ✓ x4X nie określono
  - ✓ x5x odpowiedź dotycząca statusu odbiorcy.
- Trzecia cyfra dokładniej precyzuje kod odpowiedzi

Protokoły – SMTP, POP3

Zasada działania – Protokół SMTP – kody odpowiedzi.

- Odpowiedz serwera składa się z trzech cyfr i komunikatu słownego
  - ✓ 211 Odpowiedź na polecenie HELP, informacja o stanie systemu.
  - ✓ 214 Komunikat pomocy dla użytkownika.
  - ✓ **220 Informacja o gotowości usługi.**
  - ✓ 221 Zamykanie połączenia.
  - ✓ **250 Powodzenie żądanej operacji.**
  - ✓ 251 Użytkownik nie odnaleziony, przekazywanie na adres <ścieżka>.
  - ✓ **354 Rozpoczęcie przyjmowania treści wiadomości e-mail.**
  - ✓ 421 Usługa nie jest dostępna.
  - ✓ 450 Operacja nie została podjęta, skrzynka jest w użyciu.
  - ✓ 451 Operacja przerwana, błąd lokalny.
  - ✓ 452 Operacja nie została podjęta, brak miejsca na dysku.

Protokoły – SMTP, POP3

Zasada działania – Protokół SMTP – kody odpowiedzi.

- Odpowiedz serwera składa się z trzech cyfr i komunikatu słownego
  - ✓ 500 Polecenie nie zostało rozpoznane lub ma niepoprawną składnię.
  - ✓ 501 Błąd w składni parametrów lub opcji polecenia.
  - ✓ **502 Polecenie nie obsługiwane.**
  - ✓ **503 Niewłaściwa kolejność przesyłanych poleceń.**
  - ✓ **504 Parametr polecenia nie jest obsługiwany.**
  - ✓ 550 Operacja nie została podjęta, skrzynka nie jest dostępna.
  - ✓ 551 Użytkownik nie został odnaleziony.
  - ✓ 552 Operacja przerwana - przekroczone ograniczenie ilości danych.
  - ✓ 553 Operacja nie została podjęta, nazwa skrzynki nie jest dopuszczalna.
  - ✓ 554 Transakcja nie została wykonana poprawnie.

Protokoły – SMTP, POP3

Zasada działania – Protokół SMTP – przebieg sesii.

```
C :> telnet szssiz.tu.kielce.pl 25
220 szssiz.tu.kielce.pl Mailer Serwer
helo szsiz.tu.kielce.pl
250 szsiz.tu.kielce.pl expected
mail from: <jarek@szsiz.tu.kielce.pl>
250 Sender syntax OK
RCPT TO :<pawel@szssiz.tu.kielce.pl>
250 recipiant address syntax ok
DATA
354 Mail
```

Protokoły – SMTP, POP3

Zasada działania – Protokół SMTP – przebieg sesji.

```
  WITAJ KOLEGO
.
250 message accepted
quit
```

Protokoły – SMTP, POP3

Zasada działania – Odbieranie poczty.

- **Protokół urzędu pocztowego** – (POP, Post Office Protocol) umożliwia lokalnemu agentowi pocztowemu ściągnięcie wiadomości pocztowych z agenta przekazywania poczty do komputera użytkownika.
  - ✓ Został on po raz pierwszy opisany w roku 1984, a cztery lata później wprowadzono aktualizację do wersji POP2.
  - ✓ Obecnie obowiązującym standardem jest POP3. Agent użytkownika protokołu POP3 łączy się z serwerem za pośrednictwem protokołu TCP/IP (zazwyczaj przez port 110).

Protokoły – SMTP, POP3

Zasada działania – Odbieranie poczty.

- **Internet Message Access Protocol** – internetowy protokół pocztowy zaprojektowany jako następca POP3. W przeciwieństwie do POP3, który umożliwia jedynie pobieranie i kasowanie poczty, IMAP pozwala na zarządzanie wieloma folderami pocztowymi oraz pobieranie i operowanie na listach znajdujących się na zdalnym serwerze.

Protokoły – SMTP, POP3

Różnice POP3 - IMAP.

- IMAP
  - ✓ Synchronizacja: Zmiany (np. kasowanie, czytanie) widać na każdym urządzeniu.
  - ✓ Miejsce: Listy cały czas leżą na serwerze.
  - ✓ Wiele urządzeń ma to samo.
- POP3
  - ✓ Pobieranie: Ściąga maila na dysk komputera lub telefonu.
  - ✓ Usuwanie: Zazwyczaj kasuje oryginał z serwera po pobraniu.
  - ✓ Jedno urządzenie: Dobre do pracy tylko na jednym sprzęcie.
  - ✓ Zaleta: Oszczędność miejsca na serwerze i dostęp offline do całej bazy.

Protokoły – SMTP, POP3

Zasada działania – Protokół POP3.

- **USER użytkownik** Przedstawienie się serwerowi.
- **PASS hasło** Podanie hasła do konta.
- **STAT** Użytkownik pyta się o stan konta.

  W odpowiedzi *+OK. 4 3456* (Liczba wiadomości rozmiar)
- **LIST nr** Pytanie o szczegóły wiadomości.
- **RETR nr** Pobranie wiadomość o numerze nr.
- **DELE nr** Usunięcie wiadomości o numerze nr.
- **TOP nr n** Serwer zwraca n linii z wiadomości o numerze nr.
- **QUIT** Zakończenie połączenia.

Protokoły – SMTP, POP3

Zasada działania – Protokół POP3 – kody odpowiedzi.

- **+OK** Komenda wykonana poprawnie
- **ERR komentarz** Komenda niewykonana poprawnie

Protokoły – SMTP, POP3

Zasada działania – Protokół POP3 – przebieg sesji.

```
C :> telnet szssiz.tu.kielce.pl 110
USER pawel
+OK.
PASS dfert1
+OK.
STAT
+OK. 1 34
RETR 1
   RECEIVED FROM jarek@szssiz.tu.kielce.pl
   WITAJ KOLEGO
QUIT
```

Protokoły – SMTP, POP3

Budowa wiadomości pocztowej.

Nagłówek	Date: Sun, 23 Feb 2022 17:43:41 +0100 From: student@student.pl To: kontrolat@student.pl Subject: Test poczty MESSAGE-id: <20220223174341.2345635>
Pusta linia	
Pusta linia	CZEŚĆ To jest treść ... Pozdrawiam

Protokoły – SMTP, POP3

Budowa wiadomości pocztowej – pola nagłówka

- **Return-Path: Adres zwrotny** – Pole jest dodawane przez ostatni system pocztowy biorący udział w przesyłaniu przesyłki. Służy ono do określenia informacji o adresie i drodze do nadawcy.
- **From: Identyfikator autora** – Pole dostarcza informacje o nadawcy. Jest dodawana przy tworzeniu przesyłki. Identyfikatorem może być adres użytkownika, procesu tworzącego list. W przypadku nie dodania pola do nagłówka wymaganym jest pole Sender.
- **Sender: Identyfikator nadawcy listu** – Pole dostarcza informacji o agencie wysyłającym list. Wprowadzono je dla odróżnienia nadawcy od autora. Jeśli From=Sender wtedy pomija się Sender.
- **Reply-To: adres** – Pole dostarcza odbiorcy informację, na jaki adres należy odpowiedzieć.

Protokoły – SMTP, POP3

Budowa wiadomości pocztowej – pola nagłówka

- **Received**: – Pole jest pomocne przy ustalaniu drogi przesyłki, ewentualnym poznawaniu przyczyny niepowodzenia w jej dostarczeniu.
  - [from domena] host wysyłający
  - [by domena] host odbierający
  - [via] droga przesyłki
  - [with] protokół
  - [id] numer przesyłki
  - [for] adres pocztowy adresat oraz data i czas

Protokoły – SMTP, POP3

Budowa wiadomości pocztowej – pola nagłówka

- **to: adresaci** – W polu umieszczamy informacje o głównych adresatach przesyłki.
- **Cc: adresaci** – W polu umieszczamy informacje, komu wysłać kopię przesyłki (carbon copy).
- **bcc: adresaci** – W polu umieszczamy informacje komu wysłać kopię przesyłki. Różni się od „Cc" tym, ze jego zawartość jest ukrywana przed adresatami (blind carbon copy).
- **Message-id: identyfikator** – Pole zawiera unikalny identyfikator bieżącej wysyłki listu. Jest dodawana przez system pocztowy nadawcy.
- **in-Reply-To:** identyfikator – W polu występuje identyfikator (Message-Id) listu na który ten jest odpowiedzią

Protokoły – SMTP, POP3

Budowa wiadomości pocztowej – pola nagłówka

- **Keywords:** – Słowa kluczowe pole może zawierać pola kluczowe lub frazy specyfikujące zawartość listu.
- **Subject: temat** – Krótki opis tematyki zawartej w liście.
- **Comments: uwagi** – Pole umożliwia dodanie do przesyłki swoich uwag bez naruszania jej integralności.
- **Encrypted: technologia** – Pole zawiera informacje o zastosowanym oprogramowaniu, algorytmie szyfrującym.

Załączniki

```
Content-Type: multipart/mixed;
  boundary="----=_NextPart_000_0011_01C87AFD.827BDFD0"
------=_NextPart_000_0011_01C87AFD.827BDFD0
 Content-Type: text/plain; format=flowed; charset=iso-8859-2;
   reply-type=original
 Content-Transfer-Encoding: 7bit

TRESC WIADOMOSC
------=_NextPart_000_0011_01C87AFD.827BDFD0
Content-Type:  text/plain;  format=flowed;  name=a1.txt;  reply-
type=original
Content-Transfer-Encoding: 7bit
Content-Disposition: attachment;       filename="a1.txt"

To jest zawartosc pliku
------=_NextPart_000_0011_01C87AFD.827BDFD0
```

Protokół ESMTP

Protokół ESMTP jest rozszerzoną wersją standardowego protokołu SMTP. Poszerza on standardowy protokół o możliwość autoryzacji podczas wysyłania wiadomości i obsługę standardu MIME. Do zasadniczych zmian protokołu należy obsługa komendy EHLO. Kiedy system łączy się z programem MTA, może użyć zmodyfikowanego polecenia HELO czyli EHLO. Jeżeli MTA obsługuje ESMTP, odpowiedzią będzie lista zrozumiałych dla niego poleceń. Jeżeli serwer nie obsługuje ESMTP, odpowiada komunikatem o błędzie (500 - polecenie nie rozpoznane). Wówczas stacja wysyłająca powraca do standardowego SMTP.

Protokoły – ESMTP

Protokół IMAP (port 143)

Wiadomości są przechowywane na serwerze zdalnym. Użytkownicy mogą logować się za pomocą wielu klientów poczty, a także czytać te same wiadomości. Wszystkie zmiany wprowadzone w skrzynce pocztowej zostaną zsynchronizowane na wszystkich urządzeniach, a wiadomości będą jedynie usuwane na serwerze, jeśli użytkownik skasuje daną wiadomość e-mail.

- Użytkownik może być zalogowany jednocześnie na wielu komputerach i urządzeniach.
- Archiwum Twojej poczty jest synchronizowane i zapisywane na serwerze, aby wszystkie połączone urządzenia miały do niego dostęp.
- Wiadomości wysłane i odebrane są przechowywana na serwerze, chyba że użytkownik trwale je usunie.

Protokoły – serwer pocztowy

Serwery pocztowe

- Cechy serwera
  - ✓ Obsługa wielu domen
  - ✓ Zakładanie dowolnej ilość bez szelowych kont.
  - ✓ Ustalenie zakresów adresów ip, z których można się połączyć się do wysyłania poczty.
  - ✓ Pełne logowanie wszystkich sesji użytkowników.
  - ✓ Przekierowanie wiadomości
  - ✓ Współpracować ze skanerami antywirusowymi
  - ✓ Być darmowy